SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

ASSIGNMENT – 1

SOFTWARE TESTING (CSA3714)

Particulars	Details
Programme	B.E. / B.Tech
Course Code	CSA3714
Course Name	Software Testing
Assignment Topic	Evaluation of Software Testing Practices in the MediQueue Application
Student Name	Divyadharshini A
Register Number	192411118

ASSIGNMENT OVERVIEW

This report analyses the MediQueue hospital outpatient appointment application using the ten incidents and the defect data supplied in the assignment. It evaluates the original testing practices, identifies defect origins and consequences, and recommends a practical quality improvement process. Task 12 adapts that process to two-week releases and integration with the national digital health ID system.

Important interpretation of the supplied data: the table records the number of defects introduced in each phase, the number detected in each phase, and an average cost to fix a defect at that phase. It does not map each individual defect to its exact introduction phase and later detection phase. Therefore, calculations below distinguish the observed cost calculation from illustrative savings estimates and state the assumptions used.

CASE STUDY FACTS

- MediQueue is a mobile and web application used for outpatient appointments, consultation-fee payments, token generation and prescription viewing.
- Approximately 6,000 patients use the application every day.
- The application was developed in eight months by five developers, with no separate tester.
- Developers tested their own code shortly before release; approval was based mainly on a management demonstration.
- Ten incidents were reported after release, and the defect data records 100 defects, of which 60 were detected in production.

TASK 1 – EVALUATE TESTING AS AN ENGINEERING ACTIVITY

1. Introduction

Software testing is an engineering activity when it is systematic, planned, measurable, repeatable and connected to defined quality objectives. It is not merely an attempt to run the application and see whether it appears to work. Testing should provide objective evidence that requirements have been implemented correctly and that the software is sufficiently reliable for its intended use.

MediQueue was developed by five developers over eight months without a separate tester. Developers tested their own code just before release, and the release was approved after a demonstration to hospital management. This approach did not provide adequate evidence of quality. A demonstration can show selected successful paths, but it cannot establish that concurrent bookings, unusual patient data, payment boundaries, privacy controls and performance conditions work correctly.

Therefore, MediQueue did not systematically engineer quality into the product. The team relied mainly on informal self-checking and a successful demonstration.

2. Characteristics of testing as an engineering activity

A. Planning

Planning determines the scope, objectives, resources, responsibilities, schedule, risks, test levels, test data and entry/exit criteria for testing. MediQueue had no separate tester and no evidence of a structured test plan. Testing was delayed until just before release. This made it likely that important cases were overlooked. For example, I1 (duplicate token numbers) required a planned concurrency test in which two patients book the same doctor at the same time. I2 also required a planned test of the hospital's weekly operating calendar.

B. Measurement

Measurement uses objective indicators such as requirements coverage, test execution status, pass/fail rate, defect leakage, defect removal efficiency, severity distribution and response time. MediQueue's release decision was based on a demonstration rather than measurable acceptance criteria. The supplied data shows that 60 of 100 defects were detected after release. This is strong evidence that pre-release defect detection was inadequate. The 25-second schedule-screen load time in I10 also shows why response-time targets should have been measured.

C. Standards

Standards establish consistent methods for requirements documentation, design reviews, coding, test-case writing, defect reporting, security and release approval. In MediQueue, the Sunday closure rule was not documented (I2), and the GST rule was misunderstood (I3). These are signs that business rules and acceptance criteria were not governed by a consistent requirements standard. A standard for input validation could also have required the patient-name length limit to be handled safely rather than causing a crash (I4).

D. Repeatability

Repeatability means that another tester, or the same tester later, can execute a documented test under controlled conditions and obtain a consistent result. Test steps, test data, environment details and expected results must be recorded. In I8, a refund test was marked Pass because the expected value was wrong. This demonstrates that the test oracle—the source used to decide the correct result—was not independently verified. Reusable test cases and reviewed expected results would make the test more reliable.

3. Missing characteristics and their impact

Characteristic	Evidence in MediQueue	Effect on quality
Planning	No independent tester; testing performed just before release.	Concurrency, boundary, business-rule and integration scenarios were likely missed.
Measurement	Approval based on a demonstration, with no stated quality metrics.	The team could not objectively judge coverage, defect leakage or performance.

Standards	No formal requirements review; GST and Sunday rules were misunderstood or undocumented.	Different team members could interpret business rules inconsistently.
Repeatability	Refund test passed against an incorrect expected value.	Test results were unreliable and defects could be incorrectly closed.

4. What an engineered testing approach should have looked like

1. Requirements review: clarify business rules such as Sunday closures, GST, senior-citizen eligibility and maximum patient-name length. Each rule should have a measurable acceptance criterion.
2. Test planning: prepare a test plan covering functional, integration, security, usability, performance, concurrency and regression testing. Assign responsibilities and define release criteria.
3. Design review: inspect token-generation logic, database transactions, prescription-file handling, access control and system architecture before coding is completed.
4. Unit testing: test individual components such as fee calculation, age eligibility, name validation and token generation.
5. Integration testing: verify data exchange among booking, payment, token, prescription and SMS services.
6. System and non-functional testing: test complete patient and doctor workflows, concurrent users, privacy, response time and reliability.
7. Regression testing: rerun relevant existing tests after every correction or change.
8. Defect reporting and release evaluation: record each defect, classify severity and priority, retest fixes, and release only when agreed acceptance criteria are met.

5. Evaluation and conclusion

The MediQueue approach cannot be considered a mature engineering-based testing process because planning, measurement, standards and repeatability were weak or absent. The 60 production-detected defects and the specific incidents show that a successful demonstration was not sufficient evidence of correctness. A planned, measurable and repeatable testing process throughout the life cycle would have detected more defects before patients were affected, reduced correction costs and improved patient safety.

TASK 2 – EVALUATE THE ROLE OF PROCESS IN SOFTWARE QUALITY

1. Importance of a defined process

Software quality depends on both capable people and a defined process. Skilled developers are important, but individual skill cannot replace clear requirements, reviews, independent testing, controlled test data, measurable exit criteria and defect tracking. A defined process makes quality activities consistent and ensures that important work is not forgotten when schedules are tight.

2. Would better developers alone have prevented the incidents?

No. Better developers might have reduced some coding defects, but they would not necessarily have prevented I2 because the Sunday closure rule was never written in the requirements. They might also have repeated the incorrect GST interpretation in I3 if the business rule was unclear. I8 could still occur if a test case used an incorrect expected refund value. I9 could occur when an external SMS provider changes its API key, even if the application was originally coded by highly skilled developers. I6 also requires deliberate design, privacy controls and concurrent-access testing.

3. Process maturity assessment

Using a typical five-level Test Maturity Model (TMM) view, MediQueue is closest to Level 1 – Initial. At this level, testing is informal, reactive and dependent on individual effort. The evidence is that developers tested their own code just before release, there was no independent tester, no formal review process was described, and defects were largely discovered by patients after release.

TMM level	General meaning	MediQueue position
Level 1 – Initial	Ad hoc testing; success depends on individual effort.	Current practice is closest to this level.
Level 2 – Defined	Basic testing policies, planning and monitoring are established.	Recommended first target.
Level 3 – Integrated	Testing is integrated into the development life cycle.	Longer-term target.
Level 4 – Managed and measured	Quantitative quality measures guide decisions.	Future improvement.
Level 5 – Optimization	Continuous improvement and defect prevention are institutionalised.	Long-term maturity goal.

Conclusion: quality is not mainly the result of hiring better developers. It is the result of capable people working within a reliable process. MediQueue should first establish a defined process and then improve its maturity through measurement and continuous prevention.

TASK 3 – EVALUATE TESTING AS A PROCESS

1. Testing throughout the life cycle

Testing should begin when requirements are created and continue through design, coding, integration, system testing and maintenance. It includes planning, test analysis and design, execution, evaluation, defect reporting and closure. If testing had been treated as a continuous process in MediQueue, defects could have been detected closer to the phase in which they were introduced.

2. Testing process activities

Activity	Application to MediQueue
Planning	Define scope, risks, resources, test levels, schedule and release criteria.
Analysis and design	Derive test conditions and cases from requirements; prepare valid and invalid data and expected results.
Execution	Run unit, integration, system, security, performance and regression tests.
Evaluation	Compare actual results with reviewed expected results; measure coverage and defect leakage.
Reporting and closure	Log defects, assign severity/priority, retest fixes, close defects and report release readiness.

3. Verification and validation across development phases

Verification asks whether work products are being built correctly according to specifications. It includes reviews, inspections and technical checks. Validation asks whether the completed system meets users' actual needs. Both should be applied throughout the life cycle.

Development phase	Verification activities	Validation / testing activities
Requirements	Review completeness, consistency, testability and business rules.	Confirm workflows and acceptance criteria with hospital staff and representative users.
Design	Inspect architecture, database schema, concurrency control and security design.	Prototype important workflows and validate design choices with users.
Coding	Code reviews, static analysis and unit-test review.	Execute unit tests against expected behaviour.
Integration	Review interface contracts and data mappings.	Test booking-payment-token-prescription-SMS interactions.
System / release	Check traceability, test evidence and release criteria.	Validate end-to-end workflows, performance, security and usability.

4. Phase in which each incident should have been detected

Incident	Best detection phase	Reason
I1 – Duplicate token	Design, integration and concurrency testing	Token allocation must be atomic and unique under simultaneous bookings.
I2 – Sunday appointments	Requirements review and acceptance testing	Hospital closure rule should be documented and verified.
I3 – Incorrect fee/GST	Requirements review, design and unit testing	GST rule and displayed/charged amount must be consistent.
I4 – Long name crash	Design, coding and boundary-value testing	Input length and safe validation should be handled.

I5 – Senior-citizen priority	Requirements review and boundary-value testing	Age eligibility must include a patient who turns 60 on appointment day.
I6 – Wrong prescription details	Design, code review, integration and security testing	Patient data must be isolated and concurrent file generation must be safe.
I7 – Incorrect manual menu	Documentation review and usability testing	Instructions must match the actual application interface.
I8 – Wrong refund oracle	Test design and test-case review	Expected refund value must be independently checked against the approved rule.
I9 – SMS API key change	Integration and configuration testing	External credentials and provider changes need monitoring and regression tests.
I10 – Slow schedule screen	Design review and performance testing	Database queries, data volume and network response should be measured.

5. Cost of defects detected late

The supplied table gives the following average cost to fix one defect at the phase where it is detected:

Detection phase	Defects detected	Cost per defect	Calculated cost
Requirements	2	₹500	2 × 500 = ₹1,000
Design	3	₹1,500	3 × 1,500 = ₹4,500
Coding	15	₹3,000	15 × 3,000 = ₹45,000
Testing	20	₹8,000	20 × 8,000 = ₹1,60,000
Production	60	₹40,000	60 × 40,000 = ₹24,00,000
Total	100	—	₹26,10,500

Calculation: Total recorded correction cost = (2 × ₹500) + (3 × ₹1,500) + (15 × ₹3,000) + (20 × ₹8,000) + (60 × ₹40,000) = ₹26,10,500.

Production accounts for ₹24,00,000 of the calculated total, which is approximately 91.9% of the total correction cost (₹24,00,000 ÷ ₹26,10,500 × 100). Only 60% of defects were detected after release, yet they account for the overwhelming majority of the estimated correction cost. This demonstrates the financial impact of late detection.

6. Likely saving from requirements and design reviews

The table records 30 defects introduced in requirements and 22 introduced in design, giving 52 defects associated with these early phases. The table does not identify exactly where each of these 52 defects was later detected. Therefore, an exact saving cannot be calculated from the supplied data alone.

An illustrative upper-bound scenario can show the potential benefit. Assume that formal reviews detect all 30 requirements defects while still in the requirements phase and all 22 design defects while still in the design phase. Their early correction cost would be:

Requirements: 30 × ₹500 = ₹15,000.

Design: 22 × ₹1,500 = ₹33,000.

Total early correction cost = ₹15,000 + ₹33,000 = ₹48,000.

For comparison only, if all 52 defects escaped until production, their correction cost would be 52 × ₹40,000 = ₹20,80,000. Under that assumption, the gross cost difference would be ₹20,80,000 − ₹48,000 = ₹20,32,000.

This is an illustrative scenario, not a proven saving. It assumes all 52 defects would otherwise reach production and that reviews identify every one. It excludes the time and cost of conducting reviews, and it does not account for defects that might be found in design, coding or testing. Actual savings would depend on review effectiveness and the later detection phase.

7. Limits of the estimate and conclusion

The data provides phase totals rather than a defect-by-defect traceability matrix. It does not identify the introduction phase and detection phase for each individual defect, nor does it state review effort, rework hours, operational losses or patient impact. The cost figures are average values and may not represent every defect equally. Despite these limitations, the data clearly shows that production detection is extremely expensive. MediQueue should shift testing left through requirements reviews, design inspections, unit tests and early integration tests, while continuing system and acceptance testing before release.

TASK 4 – APPLY AND CRITIQUE BASIC DEFINITIONS

1. Meaning of key testing terms in MediQueue

Term	Meaning applied to MediQueue
Error	A human mistake, such as misunderstanding the GST rule or incorrectly interpreting the senior-citizen eligibility requirement.
Fault / defect	A flaw in a requirement, design, code, test case or document, such as incorrect fee logic or unsafe prescription-file handling.
Failure	An observable incorrect behaviour when the application runs, such as charging ₹550 after displaying ₹500.
Test case	A documented set of preconditions, input data, execution steps and expected results; for example, two patients booking the same doctor at the same instant.
Test oracle	The trusted source used to decide the expected result, such as the approved hospital fee policy or refund calculation rule.

Test bed	The controlled environment, including application build, database, devices, network, test accounts, test data and connected services used to execute tests.

2. Error, fault and failure in incidents I3, I5 and I6

Incident	Error (human mistake)	Fault / defect in work product	Failure observed
I3 – Fee mismatch	GST rule was misunderstood by the team.	Fee calculation or display/charging logic does not apply the correct tax rule consistently.	Booking page shows ₹500 but ₹550 is charged.
I5 – Senior priority	Eligibility was interpreted without considering the patient's age on the appointment date.	Age calculation or priority condition excludes a person who turns 60 on that date.	Eligible patient does not receive senior-citizen priority.
I6 – Prescription mix-up	Design or coding failed to account for simultaneous saves and patient-data isolation.	Shared or unsafe file/state handling allows one patient's details to be used in another patient's PDF.	A prescription PDF displays the previous patient's details.

The exact internal coding cause of I6 is not specified in the case. Shared temporary files or unsynchronised shared state are possible explanations, but the confirmed evidence is the cross-patient data mix-up during simultaneous saves.

3. Why terminology confusion contributed to I8

I8 shows that the team marked a refund test as Pass even though the expected value was wrong. If the team uses 'bug', 'error' and 'failure' interchangeably, it may fail to distinguish a human mistake in writing a test from a defect in the test case and an actual failure in the application. The team may also close a defect without checking the authority behind the expected result. Correct terminology helps identify whether the problem lies in the requirements, software, test design or test execution.

4. Need for a common glossary

MediQueue needs a shared glossary so developers, testers, hospital staff and managers use the same terms. It should define error, defect/fault, failure, test case, test oracle, test bed, severity, priority, retest, regression test, verification, validation and acceptance criteria. Each entry should include a short MediQueue-specific example and identify who approves authoritative business rules. This will improve defect reports, communication, root-cause analysis and test accuracy.

Conclusion: terminology is not merely a matter of words. Clear distinctions help the team locate the source of a problem, choose the correct corrective action and avoid falsely declaring a test successful.

TASK 5 – EVALUATE THE APPLICATION OF TESTING PRINCIPLES

The claim that 'we tested everything and found no errors, so the software was correct' is not valid. Software testing follows principles that explain why testing cannot prove the complete absence of defects.

Testing principle	Meaning	MediQueue evidence and evaluation
Testing shows the presence of defects, not their absence	Passing tests only show that the tested cases passed; they do not prove that no defects remain.	The management demo showed no visible error, but I1, I3 and I6 appeared in real use. The demo could not prove the whole application was correct.
Exhaustive testing is impossible	Testing every input, path, user combination and timing condition is usually impractical.	With about 6,000 daily users, every possible concurrent booking and patient-data combination cannot be tested. Risk-based selection and automation are needed.
Early testing saves time and cost	Reviews and tests should begin as soon as requirements and designs are available.	I2 and I3 could have been identified during requirements review. The production defect cost of ₹40,000 is much higher than the ₹500 requirements-phase cost.
Defects cluster together	A small number of components often contain a large proportion of defects.	Booking, payment, token generation and prescription handling are high-risk areas. Testing effort should focus on these critical workflows.
Pesticide paradox	Repeating the same tests eventually finds fewer new defects; tests must be reviewed and expanded.	A repeated management demo may continue to miss concurrency, boundary and integration defects. New test scenarios are required.
Absence-of-errors fallacy	A system can be technically free of known defects yet still fail to meet user needs.	Even if the app had no coding errors, accepting Sunday appointments (I2) or failing to apply hospital priority rules (I5) would still make it unsuitable.

The principle that would have had the largest overall effect is early testing. MediQueue's requirements contained unclear or missing rules, and the defect data shows that early correction is much cheaper than production correction. Requirements reviews, design inspections and early test-case preparation would have prevented several defects from moving into later phases. However, early testing must be combined with independent testing and risk-based coverage; it cannot replace later system testing.

Conclusion: a successful demonstration and a set of passing tests do not prove correctness. Testing reduces risk by finding defects and providing evidence, but it cannot guarantee that every defect has been removed.

TASK 6 – EVALUATE THE TESTER'S ROLE

1. Tester responsibilities across the life cycle

A tester is responsible for more than executing test cases at the end of development. The tester reviews requirements for ambiguity, identifies risks, prepares test conditions, designs test cases and data, checks

expected results, executes tests, reports defects, retests corrections, performs regression testing and communicates product quality to the release decision-makers. In a healthcare application, the tester must also consider privacy, security, accuracy, availability and patient safety.

2. Advantages and limitations of developer self-testing

Developer self-testing – advantages	Developer self-testing – limitations
Developers understand the code and can quickly test individual components.	Developers may share the same assumptions that caused the original defect.
Unit testing can detect coding errors early and is relatively inexpensive.	They may focus on expected workflows and overlook unusual user behaviour.
Immediate feedback helps developers correct defects quickly.	They may not independently verify business rules or test oracles.
It supports automated tests in the development pipeline.	It does not replace independent system, usability, security and acceptance testing.

3. Incidents an independent tester could likely have detected

Incident	Likely testing stage and test activity
I1	Integration/system testing with simultaneous booking and concurrency tests.
I2	Requirements review and acceptance testing against the hospital calendar.
I3	Requirements review and payment integration tests comparing displayed and charged totals.
I4	Boundary-value testing with names around and above 40 characters.
I5	Boundary-value tests for a patient turning 60 on the appointment date.
I6	Concurrent prescription-generation tests and patient-data isolation/security checks.
I7	Documentation review and usability testing of cancellation instructions.
I8	Independent review of the refund test oracle and expected result.
I9	Integration and configuration tests for changed API credentials and provider errors.
I10	Performance testing using realistic doctor schedules, data volumes and hospital Wi-Fi conditions.

4. Qualities required of a healthcare software tester

- Attention to detail and strong analytical thinking, especially for fees, eligibility and patient identity.
- Understanding of test design techniques such as boundary-value analysis, equivalence partitioning, decision tables and concurrency testing.
- Awareness of privacy, confidentiality, access control and secure handling of medical information.
- Clear communication and accurate defect documentation.
- Independence, curiosity and willingness to challenge assumptions.

- Ability to prioritise defects according to patient safety, data integrity and operational impact.

Conclusion: an independent tester would probably have reduced the number of defects reaching patients, but would not automatically have prevented every defect. The tester must be involved from requirements through release, while developers remain responsible for unit testing and code quality. Quality is a shared responsibility, supported by independent evaluation.

TASK 7 – TRACE THE ORIGINS OF DEFECTS

Defect origin refers to the life-cycle phase in which a mistake or flaw was introduced. The phase where a defect is detected may be much later than its origin. The following classification identifies the most likely origin using the evidence in the case. Where the internal implementation is not described, the classification is a reasoned diagnosis rather than a confirmed code-level finding.

Incident	Most likely origin	Supporting evidence and reasoning
I1 – Duplicate token	Design / coding	Token allocation did not safely handle two simultaneous bookings. The likely weakness is in uniqueness, transaction or concurrency design.
I2 – Sunday booking	Requirements	The hospital's Sunday closure rule was never written in the requirements. The system could not enforce an undocumented rule reliably.
I3 – Fee mismatch	Requirements / design / coding	The GST rule was misunderstood, causing the displayed amount and charged amount to differ. The primary origin is unclear business-rule interpretation, implemented in the fee logic.
I4 – Long-name crash	Coding / design	The application crashes when a name exceeds 40 characters, indicating inadequate input validation, boundary handling or data-length management.
I5 – Senior priority	Requirements / coding	The eligibility rule did not correctly handle a birthday on the appointment date. The boundary condition was either omitted from the requirement or incorrectly implemented.
I6 – Wrong prescription details	Design / coding	Simultaneous saves result in another patient's details appearing in a PDF. This indicates a data-isolation or concurrency weakness in prescription generation.
I7 – Non-existent menu	Documentation / configuration control	The manual refers to 'My Visits', which is absent from the application. Documentation was not checked against the released interface.
I8 – Incorrect refund expected value	Testing	The test was marked Pass against a wrong expected result. The test oracle or test-case review process was defective.
I9 – SMS stops after API key change	Integration / environment / configuration	The external provider's credential change was not handled or detected. Configuration management and integration resilience were inadequate.
I10 – Slow schedule screen	Design / coding / environment	A 25-second load time suggests inefficient queries, data retrieval or rendering, potentially worsened by the hospital Wi-Fi environment. Performance

		requirements were not adequately tested.

Overall pattern and recommended investment

The incidents do not come from coding alone. They include missing or misunderstood requirements (I2, I3 and I5), design and concurrency weaknesses (I1 and I6), coding and input-handling issues (I4), testing weaknesses (I8), documentation mismatch (I7), external configuration/integration problems (I9) and performance problems (I10). This pattern shows that adding more code checks alone will not solve the quality problem.

The hospital should invest first in requirements clarification and review, design inspections for high-risk data flows, and systematic test design. It should then strengthen automated unit, integration, concurrency, regression and performance testing. Documentation and external-service configuration must also be controlled. Root-cause categories should be recorded in the defect repository so that improvement effort is based on evidence.

TASK 8 – EVALUATE DEFECT CLASSES AND SEVERITY

1. Classification scheme

A useful defect classification scheme separates defects by where they originate and by their technical nature. Requirements/specification defects include missing, ambiguous or incorrect business rules. Design defects include architecture, data flow, concurrency and security weaknesses. Coding defects include incorrect logic, crashes and input-handling errors. Testing defects include incorrect test cases, test data or expected results. Documentation defects include inaccurate user instructions. Environment and integration defects include configuration, credentials, network and external-service compatibility problems.

2. Severity and priority of all incidents

Severity describes the impact of a defect if it occurs. Priority describes how urgently the organisation should fix it, considering risk, frequency, workaround, business impact and release plans. The ratings below are proposed for MediQueue and should be confirmed by the hospital's clinical and operational owners.

Incident	Defect class	Suggested severity	Suggested priority	Justification
I1 – Duplicate token	Design / coding – concurrency	High	P1 – Urgent	Two patients may be assigned the same doctor's token, disrupting access and appointment order.
I2 – Sunday booking	Requirements / specification	Medium	P2 – High	Patients can book when the department is closed, causing inconvenience and operational disruption.
I3 – Fee	Requirements / coding –	High	P1 – Urgent	Incorrect charging affects financial accuracy,

mismatch	payment calculation			trust and possible refund obligations.
I4 – Long-name crash	Coding – input validation	Medium	P2 – High	Some patients cannot use the application with longer names; a workaround may exist but access is affected.
I5 – Senior priority missing	Requirements / coding – eligibility logic	High	P1 – Urgent	A promised priority rule is not applied to eligible older patients and may affect access to care.
I6 – Wrong patient prescription	Design / coding – data isolation and privacy	Critical	P1 – Immediate	Another patient's medical information may be disclosed or associated with the wrong patient, creating serious privacy and clinical risks.
I7 – Incorrect manual menu	Documentation	Low to Medium	P2 – High	Patients receive incorrect cancellation instructions. It is less directly harmful than a data mix-up but can prevent timely cancellation.
I8 – Wrong refund oracle	Testing – test case / expected result	High	P1 – Urgent	A false Pass can allow incorrect refund behaviour to remain undetected and weakens confidence in test evidence.
I9 – SMS failure after API key change	Integration / environment – configuration	Medium	P2 – High	Reminders stop, increasing missed appointments and support workload. Booking itself may still be available.
I10 – 25-second schedule load	Design / coding – performance	Medium	P2 – High	Slow doctor workflow wastes staff time and may delay operations, especially during busy periods.

3. Why severity and priority can differ

Severity and priority are related but not identical. I1 has high severity because duplicate tokens can disrupt patient access and appointment order; it also deserves urgent priority because the defect can affect simultaneous bookings and should be fixed before further releases. I7 may have lower technical severity because it is a documentation mismatch rather than a direct data-integrity failure. However, it can still receive high priority if many patients rely on the incorrect cancellation instructions or if cancellation deadlines are approaching. A low-severity defect can therefore be urgent because of its frequency or business timing, while a severe but rare defect may require immediate risk containment even if a permanent fix takes longer.

4. Defect class to target first

MediQueue should first target requirements/specification and high-risk design defects, while immediately containing the critical I6 data-isolation risk. The case shows that missing or misunderstood business rules contributed to I2, I3 and I5, while concurrency and data handling contributed to I1 and I6. Correcting

requirements and design early prevents defects from spreading into code and tests. However, because I6 can expose another patient's prescription details, it must be treated as the immediate operational priority, with access controls, investigation and safe release blocking until resolved.

TASK 9 – DESIGN AND APPRAISE A DEFECT REPOSITORY

1. Proposed defect repository fields

MediQueue should use a central, searchable defect repository rather than relying on scattered emails and a basic spreadsheet. Each record should contain enough information to reproduce the problem, assess risk, trace its origin, verify the correction and learn how to prevent similar defects.

Field	Purpose and justification
Unique defect ID	Provides a permanent reference for discussion, reports and links to code changes.
Title and description	Summarises the problem and explains the observed behaviour clearly.
Module / feature	Identifies whether the defect belongs to booking, payment, token, prescription, SMS, doctor schedule or another area.
Steps to reproduce	Allows another team member to recreate the problem consistently.
Expected result and actual result	Makes the difference between correct and observed behaviour explicit.
Test data and environment	Records test account, application version, device, browser, operating system, database and network details.
Origin phase and defect class	Supports analysis of whether defects arise from requirements, design, code, testing, documentation or integration.
Severity and priority	Separates technical impact from urgency and helps the team decide what to fix first.
Reporter, owner and dates	Shows who reported and owns the issue, when it was opened, updated and resolved.
Status and resolution	Tracks New, Assigned, In Progress, Fixed, Retest, Reopened and Closed states, including the correction made.
Attachments and evidence	Stores screenshots, logs, request IDs, test results and relevant error messages.
Root cause and prevention action	Records why the defect occurred and what process change will prevent recurrence.
Retest and regression evidence	Confirms the fix works and that related features remain correct.
Release / build reference	Identifies the version in which the defect was found and the version containing the fix.

2. How the repository supports improvement

The repository supports defect analysis by allowing the team to group defects by module, origin, severity, release and root cause. For example, repeated concurrency defects in token and prescription modules would

indicate a need for design reviews and concurrency testing. Repeated requirement defects would indicate that business-rule reviews need improvement. The repository also supports process improvement by showing defect leakage, time to resolve, reopened defects and recurring causes. Prevention actions can then be assigned and checked in later releases.

3. Weaknesses of email and spreadsheet tracking

- Different people may keep different versions, resulting in missing or conflicting information.
- Emails often lack consistent reproduction steps, severity, ownership and closure evidence.
- A spreadsheet may not provide reliable workflow, audit history, notifications or links to test and code changes.
- Duplicate complaints may be counted as separate defects, while recurring root causes remain hidden.
- Sensitive patient information may be copied into emails or spreadsheets without appropriate access controls.

Conclusion: a central defect repository improves traceability, accountability, trend analysis and prevention. Access should be role-based, and patient information should be masked or excluded unless it is strictly necessary for investigation.

TASK 10 – EVALUATE DEFECT EXAMPLES AND PREVENTION STRATEGIES

The following three incidents represent different defect classes: a requirements defect (I2), a design/concurrency defect (I1) and a testing/test-oracle defect (I8). Each is paired with one main process change intended to prevent recurrence.

1. Incident I2 – Sunday appointments accepted

Root cause: the hospital's Sunday closure rule was not documented in the requirements. The application therefore had no clear requirement against which developers or testers could verify the booking calendar.

Single process change: introduce a mandatory business-rule requirements review with the hospital's outpatient operations representative before a requirement is approved. The review must confirm operating days, holidays, doctor-specific schedules and exception rules.

Cost and benefit: a review meeting and checklist require limited staff time, especially when performed before coding. The supplied data gives a requirements-phase correction cost of ₹500 per defect compared with ₹40,000 in production. The difference is ₹39,500 per defect if the same defect would otherwise be corrected in production. The review cost is not provided, so the exact net saving cannot be calculated.

2. Incident I1 – Duplicate token numbers

Root cause: token generation did not safely handle simultaneous bookings for the same doctor. A likely design weakness is the absence of an atomic allocation or database-enforced uniqueness mechanism.

Single process change: require a formal design inspection of all shared-resource and concurrent operations, with a documented concurrency test before approval. The design must define how token numbers are allocated uniquely under simultaneous requests.

Cost and benefit: design inspection and concurrency tests require additional design and test effort, but they are cheaper than handling duplicate appointments, staff confusion and patient complaints after release. The supplied table gives ₹1,500 per design-detected defect and ₹40,000 per production-detected defect, a difference of ₹38,500 per defect if it would otherwise escape to production. The case does not provide the exact cost of the review or prove that every I1-like defect would be caught.

3. Incident I8 – Refund test passed with wrong expected value

Root cause: the refund test case used an incorrect expected result, so the comparison incorrectly reported Pass. The test oracle was not verified against an approved refund policy or calculation.

Single process change: require independent review and approval of expected results for financial test cases against an authoritative business rule before execution.

Cost and benefit: the review adds a small amount of preparation time to each important financial test. It can prevent false passes and reduce the chance of incorrect refunds reaching patients. The testing-phase correction cost is ₹8,000 per defect compared with ₹40,000 in production, a potential difference of ₹32,000 per defect if it would otherwise be detected only after release. The exact net benefit depends on review effort and the number of defects prevented.

4. Best-value strategy

Requirements review is likely to provide the best overall value as a first investment because it is relatively low effort, affects the whole development process and can prevent several classes of defects before design and coding begin. The defect data assigns the lowest correction cost to requirements-phase detection (₹500) and the highest to production detection (₹40,000). However, the hospital must not delay the immediate correction of I6 or ignore concurrency and test-oracle controls. The best practical approach is to introduce lightweight reviews as a routine part of development and apply deeper reviews to high-risk features.

TASK 11 – RECOMMEND AND DEFEND A QUALITY IMPROVEMENT PLAN

1. Recommended quality improvement plan and priority order

MediQueue should adopt a risk-based, life-cycle quality process. The plan should be introduced in stages so that a team of five developers can apply it without creating an impractical amount of paperwork.

Priority	Process change	Reason and expected result

1 – Immediate	Investigate and contain critical patient-data and concurrency risks, especially I6 and I1.	Protect patient confidentiality and prevent incorrect appointment or prescription information.
2 – Immediate	Introduce requirements clarification and peer review for business rules.	Prevent missing or misunderstood rules such as Sunday closure, GST and senior-citizen eligibility.
3 – High	Require design reviews for payment, token allocation, prescription handling, data access and external interfaces.	Identify architectural and data-isolation risks before implementation.
4 – High	Develop unit and integration tests with reviewed expected results.	Detect coding and interface defects early and prevent false Pass results such as I8.
5 – High	Create a central defect repository with severity, priority, root cause and retest evidence.	Improve traceability, accountability and trend analysis.
6 – High	Run system, regression, security, concurrency and performance tests before release.	Validate complete workflows and non-functional requirements.
7 – Continuous	Review defect trends and update prevention actions after each release.	Move from reactive correction to continuous process improvement.

2. Role of testers, reviews and repository

A designated quality engineer or tester should participate from requirements analysis to release. Developers should continue unit testing and peer code review. The tester should independently challenge assumptions, create risk-based test cases, verify expected results, execute integration and system tests, coordinate regression testing and report release risks. Requirements and design reviews should include developers and the relevant hospital representative. The defect repository should be the single source of truth for defects, ownership, evidence, root causes and closure.

3. Expected effect on defect number and cost

The supplied data shows that 60 defects were detected in production and that production correction cost was ₹40,000 per defect. The calculated total cost using all phase counts and unit costs is ₹26,10,500, of which ₹24,00,000 is associated with production-detected defects. Earlier reviews and tests should reduce defect leakage and shift detection to lower-cost phases. The exact future reduction cannot be guaranteed from the available data; it should be measured over successive releases using production defect count, defect leakage rate, severity distribution, reopened defects and cost per defect.

4. Trade-offs, risks and limitations

- More reviews and testing require time and may initially slow feature delivery.
- Automation requires setup and maintenance, and unstable tests can create false alarms.
- A tester cannot test every possible input or concurrency combination; risk-based coverage is necessary.

- External services and changing requirements may introduce defects outside the team's direct control.
- A repository must protect patient information and should not store unnecessary sensitive data.
- A small team may have limited specialist capacity for security, performance and integration testing.

5. Practicality for five developers

The plan is practical if it uses lightweight controls. Each change can receive a peer review; high-risk requirements can be reviewed with a hospital representative; unit and regression tests can be automated; and one team member can coordinate testing while developers retain responsibility for code quality. The hospital may use an independent part-time tester or quality engineer for critical releases and specialist reviews. Not every feature needs the same testing depth: payment, patient identity, prescription and appointment allocation should receive the highest risk-based attention.

Conclusion: MediQueue should move from informal last-minute checking to a defined, risk-based quality process. The immediate focus must be patient-data integrity and critical business rules, followed by reviews, automated testing, defect tracking and continuous measurement. This approach is more reliable and cost-effective than simply hiring developers and expecting quality to emerge without a process.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE QUALITY PROCESS

Decision: MODIFY the original quality improvement plan into a risk-based, continuous testing and release process. The original principles—early reviews, independent testing, defect tracking and measurable release criteria—should be retained. However, they must be strengthened with automation, continuous integration, external API contract testing, privacy controls and release monitoring to support two-week releases and national digital health ID integration.

A. Impact analysis (2 marks)

1. What has changed?

- Release frequency changes from every six months to every two weeks, reducing the time available for manual testing and requiring repeatable automated regression tests.
- MediQueue must integrate with an external national digital health ID system maintained by another agency.
- Patient health records will be shared between hospitals, increasing the impact of incorrect identity matching, unauthorised access, data leakage and inaccurate records.
- The application becomes more dependent on external API availability, authentication, data formats, version changes and service-level behaviour.

2. Assumptions affected

The original plan assumed that the team could complete a relatively long testing cycle before release and that most application behaviour was under the hospital's direct control. These assumptions are no longer sufficient. A two-week cycle requires testing to occur continuously during development. External integration means the team cannot control the other agency's release schedule, API changes or outages. Sharing records also means that privacy, consent, identity matching, auditability and data integrity must become release-blocking quality concerns.

3. Why frequent releases change testing and the tester's role

With two-week releases, repeating all tests manually would be slow and inconsistent. The tester must help define acceptance criteria early, automate stable regression tests, review changes continuously, test interfaces as soon as they are available and provide rapid risk reports. The tester also needs to verify deployment configuration, database migration, rollback readiness and post-release monitoring. Testing becomes a continuous activity rather than a phase performed after development.

4. Why record sharing raises severity

A defect that previously affected one appointment or one patient screen may now expose or corrupt information shared across hospitals. Incorrect patient matching could associate a prescription or health record with the wrong person. Unauthorised access could expose confidential medical information. Incomplete or duplicated updates could affect clinical decisions. Therefore, identity, access-control, consent, audit and data-integrity defects should be treated as high or critical severity.

B. Evaluation of the original plan

1. Do the original reviews and tests fit a two-week cycle?

The original reviews and test activities remain necessary, but a purely manual sequence may not fit every two-week release. Reviews should be performed during requirement refinement and design, not postponed until the end of the sprint. Unit tests, API tests and regression tests should run automatically when code is committed. Manual exploratory testing should focus on changed features, high-risk workflows and areas where automation is insufficient. A short release-readiness review should assess test results, open defects and operational risks.

2. New defect origins from the external health ID system

New defect origin	Possible example	Required control
API contract / schema	External agency changes a field name, format or mandatory field.	Versioned API contracts, schema validation and contract tests.
Authentication / credentials	Token expiry, key rotation or authentication protocol change.	Secure secret management, expiry monitoring and credential-rotation tests.
Network / availability	External service is unavailable or responds	Timeouts, retries with limits, circuit breakers and

	slowly.	graceful error handling.
Identity matching	Two records are incorrectly linked or a patient is matched to the wrong identity.	Strong identity validation, duplicate checks and safe exception handling.
Privacy / consent	Records are shared without the required permission or access scope.	Consent validation, role-based access and privacy testing.
Data mapping / interoperability	Different hospitals interpret codes, dates or units differently.	Standardised mappings, validation rules and cross-system test data.

3. Is the original defect repository sufficient?

The original repository is a good foundation but needs additional integration fields. It should record the external system name, API endpoint or service, API/schema version, correlation ID, request/response status, authentication method (without storing secrets), timestamp, environment, external agency release reference, data mapping version and whether patient consent was verified. It should also identify whether the defect is internal, external, interoperability-related or caused by configuration. Sensitive patient data should be masked, and access to logs must be controlled.

C. Adaptation or redesign

1. Revised quality process

9. Continuous requirements and risk review: define acceptance criteria for each change, including privacy, consent, identity matching and external-service behaviour.
10. Design and API contract review: review data flows, API versions, authentication, error handling, timeouts, retries and audit requirements before implementation.
11. Automated unit and component tests: run on each code change for business rules, input validation, fee calculations, age eligibility and data handling.
12. Continuous integration: automatically build the application and execute unit, API, integration and regression tests whenever changes are merged.
13. External-system testing: use a controlled test or sandbox environment where available; validate valid, invalid, delayed, duplicated, expired and unavailable API responses.
14. Security and privacy testing: verify role-based access, least privilege, consent, encryption, audit logs, session handling and prevention of cross-patient data exposure.
15. Release-candidate testing: execute end-to-end tests for booking, payment, token, prescription and health ID workflows, along with performance and concurrency checks.
16. Release approval and deployment: use defined quality gates, backup and rollback plans, controlled configuration and documented approval for critical changes.
17. Post-release monitoring: monitor failures, API errors, response times, duplicate records, authentication failures and patient complaints; feed findings into the next cycle.

2. Revised role of the tester

The tester becomes a continuous quality partner rather than a final-stage checker. The tester works with developers and hospital representatives to define acceptance criteria, reviews API contracts, maintains automated regression tests, performs exploratory and risk-based testing, evaluates security and privacy evidence, checks release readiness and monitors production outcomes. Developers remain responsible for unit tests and code quality, while the tester provides independent challenge and risk assessment.

3. Testing principles that become more important

- Early testing: requirements and API contracts must be checked before implementation.
- Testing is context-dependent: healthcare records and patient identity require deeper controls than low-risk display features.
- Exhaustive testing is impossible: prioritise patient safety, privacy, payment, identity and integration risks.
- Testing shows the presence of defects, not their absence: automated passing tests do not prove that all integration risks are removed.
- Pesticide paradox: refresh tests when APIs, data formats and workflows change.
- Defect clustering: concentrate testing on high-risk modules such as identity matching, prescriptions, payments and external integration.

4. Classification and tracking of external defects

External integration defects should be classified separately from internal coding defects while remaining in the same central repository. Suggested categories are external API contract, authentication/credential, availability/performance, data mapping/interoperability, identity matching, consent/privacy and external configuration. Each defect should include the service version, environment, timestamp, correlation ID, masked request/response evidence, observed behaviour, expected contract behaviour, internal owner, external dependency owner, severity, priority and workaround. This allows the team to distinguish an internal integration error from an external provider outage without losing end-to-end accountability.

D. Evidence-based defence

1. Defence using defect data and costs

The original defect data records 60 production-detected defects out of 100. The calculated correction cost is ₹26,10,500, including ₹24,00,000 for production-detected defects. This demonstrates that a late, manual testing approach is financially unsuitable. Two-week releases make the risk greater because defects can be introduced and deployed more frequently unless regression and integration checks are repeatable and automated. The revised process aims to detect defects earlier and reduce production leakage.

2. Defence using defect origins and classes

The ten incidents span requirements, design, coding, testing, documentation and integration/environment causes. The external health ID connection introduces additional interface, authentication, identity, privacy and interoperability risks. Therefore, the original plan cannot simply be retained unchanged. It needs contract testing, secure configuration management, privacy testing and external-service monitoring.

3. Defence using patient safety and data privacy

I6 already demonstrates the danger of one patient's prescription details appearing in another patient's PDF. When health records are shared across hospitals, the potential consequences become more serious. Incorrect identity matching or unauthorised disclosure can affect confidentiality and clinical decisions. For this reason, identity, access control, consent, auditability and data integrity must be treated as critical quality requirements and tested before release.

4. Defence using team size and capacity

The team still has only five developers, so a large manual testing department may not be practical. The modified process is designed to be lightweight: automate repeatable tests, use peer reviews, maintain a shared defect repository, prioritise high-risk features and use an independent quality engineer or specialist support for critical security and interoperability assessments when needed. Automation requires initial investment and maintenance, but it reduces repeated manual effort across frequent releases.

5. Final decision

The final decision is to MODIFY the original quality improvement plan, not discard it. Requirements reviews, design reviews, independent testing and defect tracking should be retained. They must be strengthened with continuous integration, automated regression testing, API contract testing, security and privacy checks, controlled deployment and post-release monitoring. This decision follows from the high production defect cost, the wide range of original defect origins, the move to two-week releases, the new external dependency and the increased patient-safety and privacy risks. It is the most practical way for a small team to improve quality while releasing more frequently.

OVERALL CONCLUSION

The MediQueue case demonstrates that software quality cannot be achieved through last-minute developer self-checking or a successful demonstration alone. The incidents arose from multiple life-cycle phases, including unclear requirements, design weaknesses, coding errors, inadequate test oracles, documentation mismatch and external configuration problems. The defect data shows that late detection is particularly expensive: 60 production defects account for ₹24,00,000 of the calculated ₹26,10,500 correction cost.

The hospital should establish a defined, risk-based testing process that begins with requirements and design reviews and continues through automated unit, integration, system, security, performance and regression testing. A central defect repository should support root-cause analysis and prevention. For two-week releases and national health ID integration, the process must be modified to include continuous testing, API contract validation, privacy and identity controls, release automation and production monitoring. These changes will improve reliability, protect patient information and reduce the likelihood and cost of defects reaching patients.